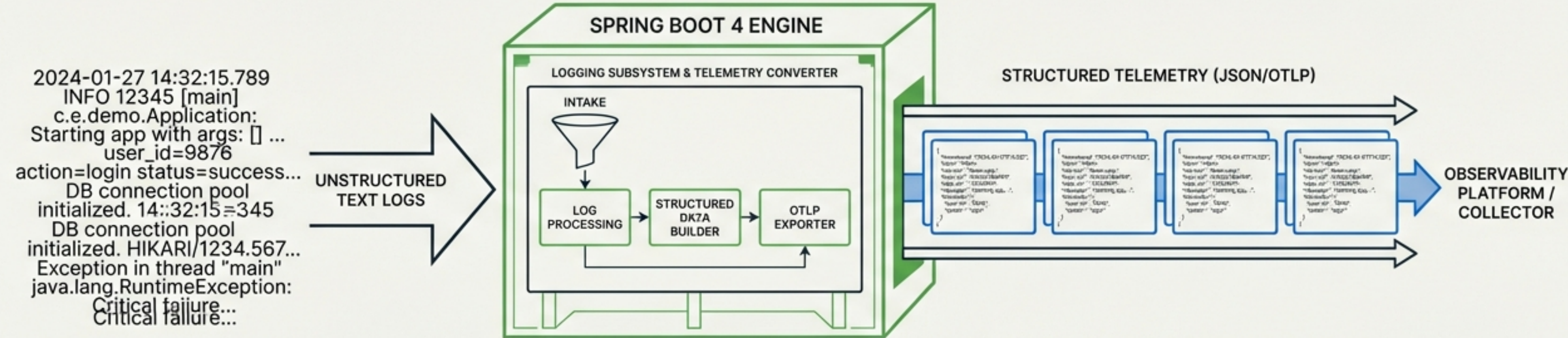


DE TEXTO A TELEMETRÍA

ARQUITECTURA Y RENDIMIENTO DEL LOGGING EN SPRING BOOT 4

TELEMETRY PIPELINE

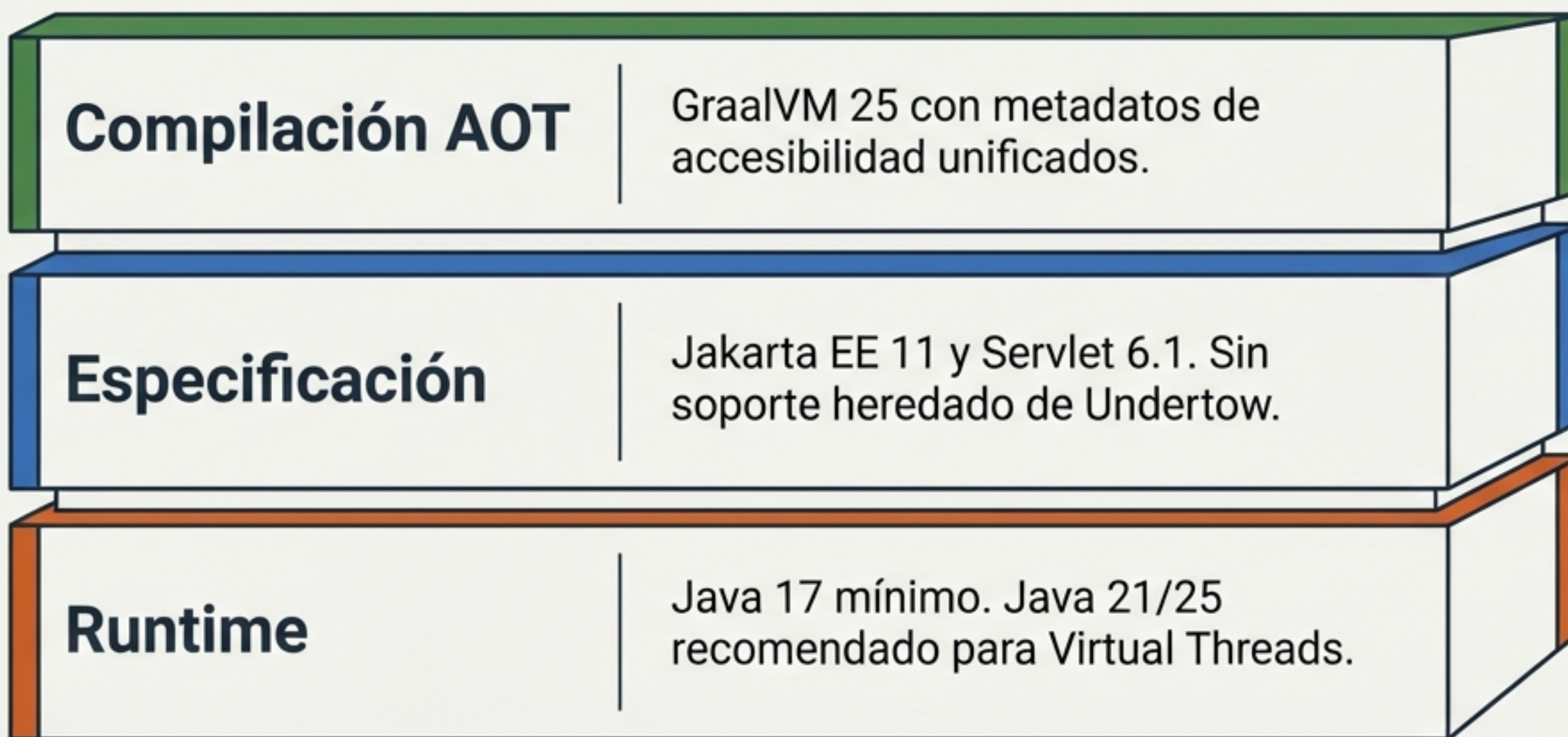


SAMPLE STRUCTURED LOG OUTPUT (JSON)

```
{
  "timestamp": "2024-01-27T14:35:01.234Z",
  "level": "Error",
  "service": "payment-service",
  "trace_id": "a7f8g9hd11j2",
  "span_id": "90123456",
  "message": "Payment processing delayed",
  "attributes": {
    "transaction_id": "TXN-SSSS",
    "delay_ms": 150
  }
}
```


Los Cimientos de la Nueva Era

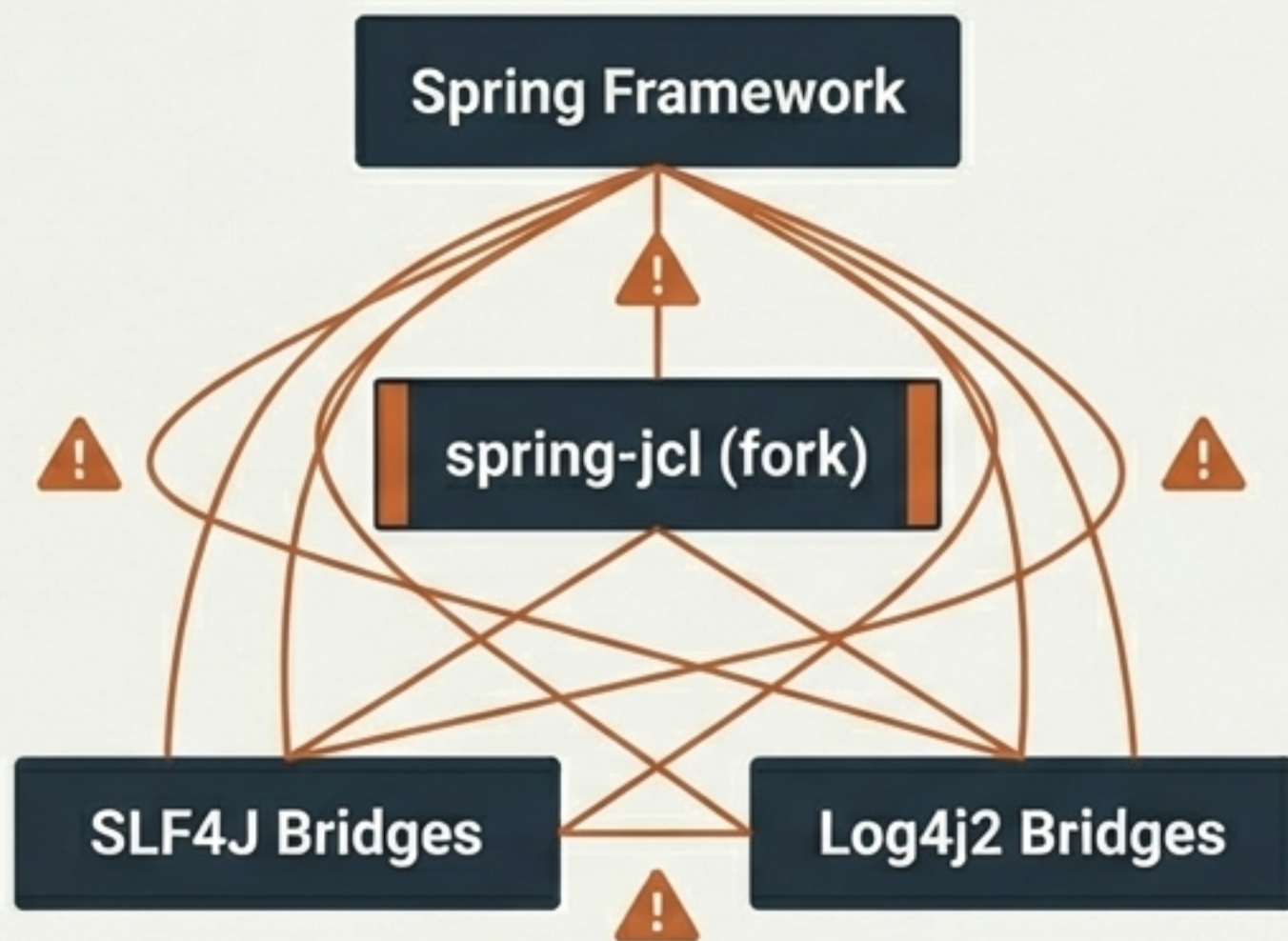
El stack de Spring Boot 4 exige una base modernizada, optimizada para cargas de trabajo cloud-native y despliegues de alta concurrencia.



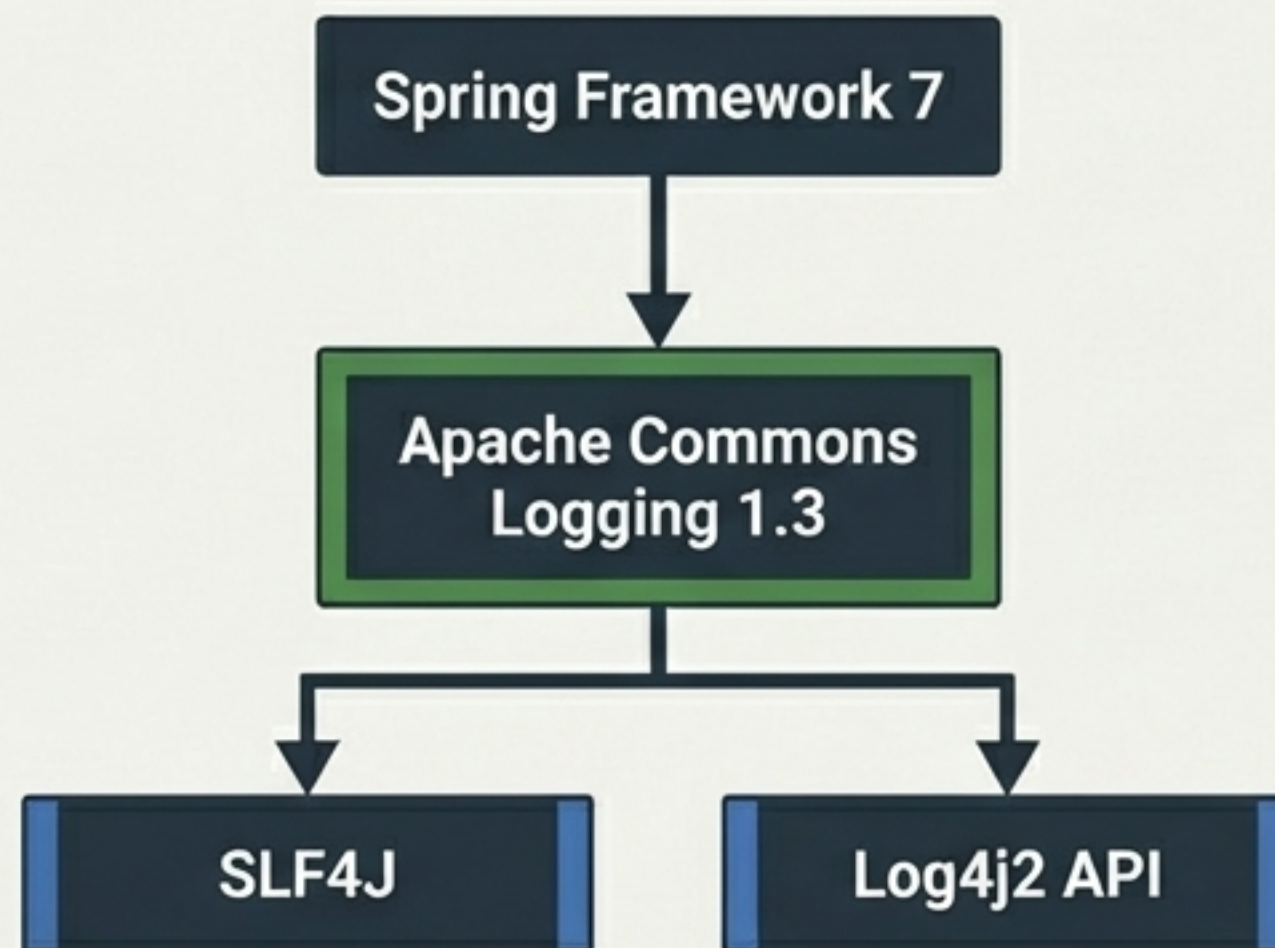
Limpieza Interna: El Fin de spring-jcl

Spring Framework 7 utiliza el verdadero Commons Logging como fachada interna, manteniendo SLF4J como la abstracción externa.

Antes (Spring Boot 3)



Ahora (Spring Boot 4)



Selección limpia y flexible por parte del usuario.

Anatomía de la Configuración Estándar

Control refinado de Logback nativo mediante application.properties.

```
logging.level.org.springframework.web=DEBUG
logging.group.db=org.hibernate,org.springframework.jdbc
logging.level.db=WARN
logging.file.name=/var/log/app.log
logging.console.enabled=false
```

Nivel por paquete

Agrupación de loggers relacionados

Control masivo vía grupo

Salida a archivo independiente

Apaga stdout para ahorrar CPU (Nuevo en v4.0)

El Conflicto: Texto vs. Telemetría

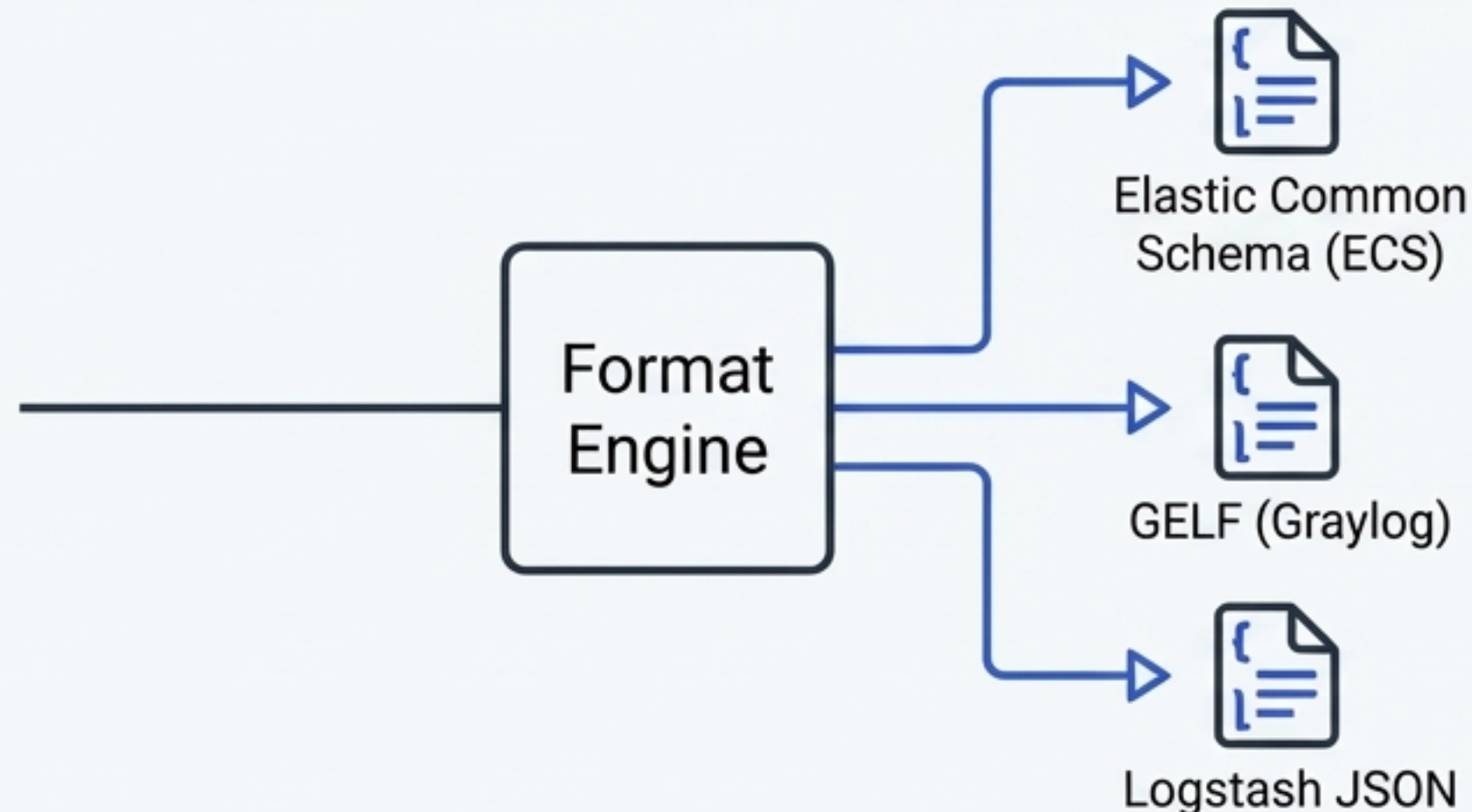
Por qué el logging tradicional se desmorona en arquitecturas distribuidas.

	Log Tradicional (Texto Plano)	Registro Estructurado (JSON)
Formato	String concatenado	Objeto JSON / Key-Value
Legibilidad Humana	Alta (Fácil en consola)	Moderada
Análisis de Máquina	Bajo (Requiere Regex)	Excelente (Nativo)
Ingesta en Agregadores	Pobre (Requiere parseo)	Óptimo (Directo)
Rendimiento de Búsqueda	Lento (Texto completo)	Rápido (Campos indexados)

Soporte Nativo para Registro Estructurado

Generación de logs legibles por máquina con cero configuración XML.

```
logging.structured.format.console=ecs
```



Enriquecimiento Automático

Inyecta automáticamente propiedades del entorno sin código adicional:

- `logging.structured.ecs.service.name`
- `logging.structured.ecs.service.version`
- `logging.structured.ecs.service.environment`

El Motor de Serialización: Jackson 3

Seguridad de hilos (thread-safety) y velocidad para cargas de alta concurrencia.

Jackson 2



ObjectMapper (Estado Mutable)

Susceptible a cuellos de botella en alta concurrencia.
Fechas en formato de timestamp numérico.

Jackson 3



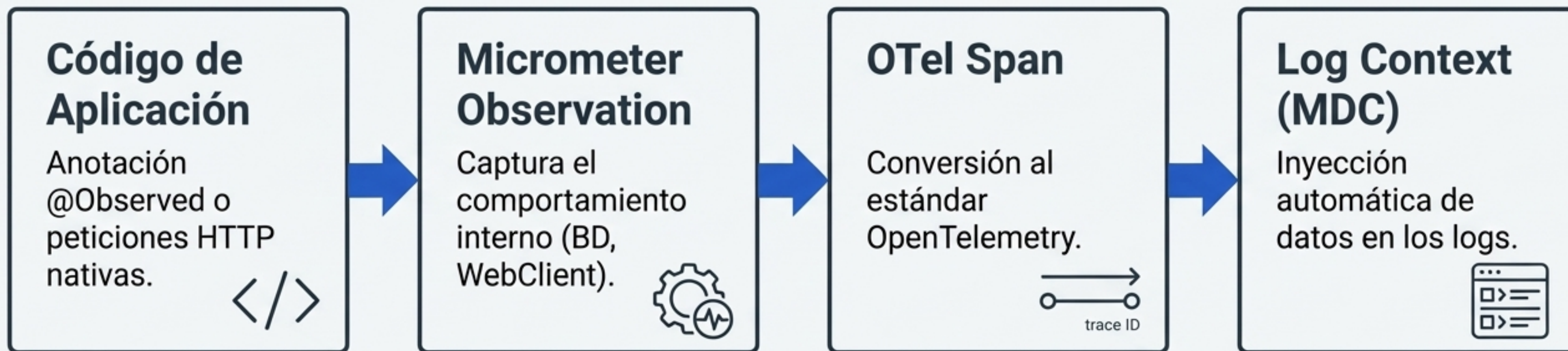
JsonMapper (Totalmente Inmutable)

Serialización segura por defecto. Fechas formateadas nativamente en cadenas ISO-8601.

Nota: Capa de compatibilidad disponible vía `spring.jackson.use-jackson2-defaults=true`

El Puente de la Observabilidad

spring-boot-starter-opentelemetry elimina la necesidad de agentes Java de terceros.

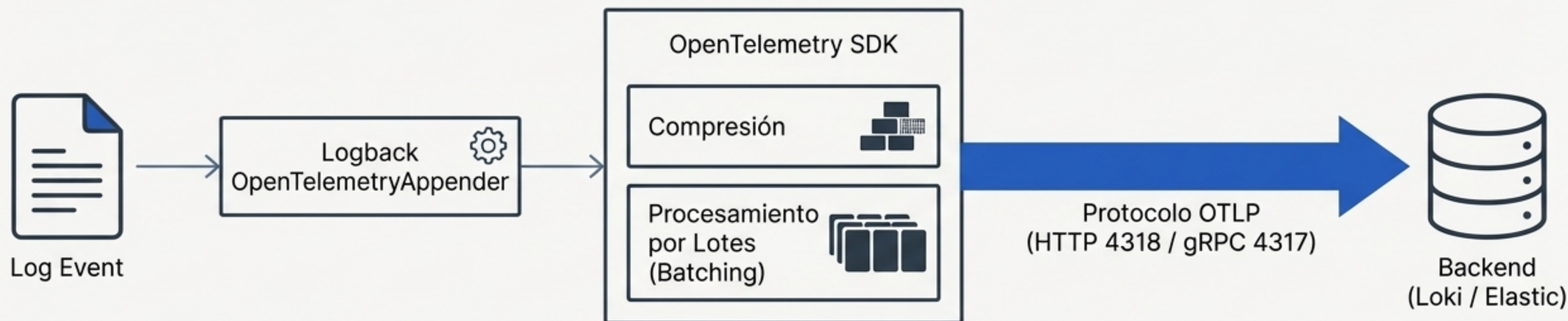


Instrumentación Automática Incluida:

- Peticiones HTTP (Spring MVC / WebFlux)
- Clientes (RestClient, WebClient)
- Operaciones de Base de Datos (JDBC)

Exportación Directa vía OTLP

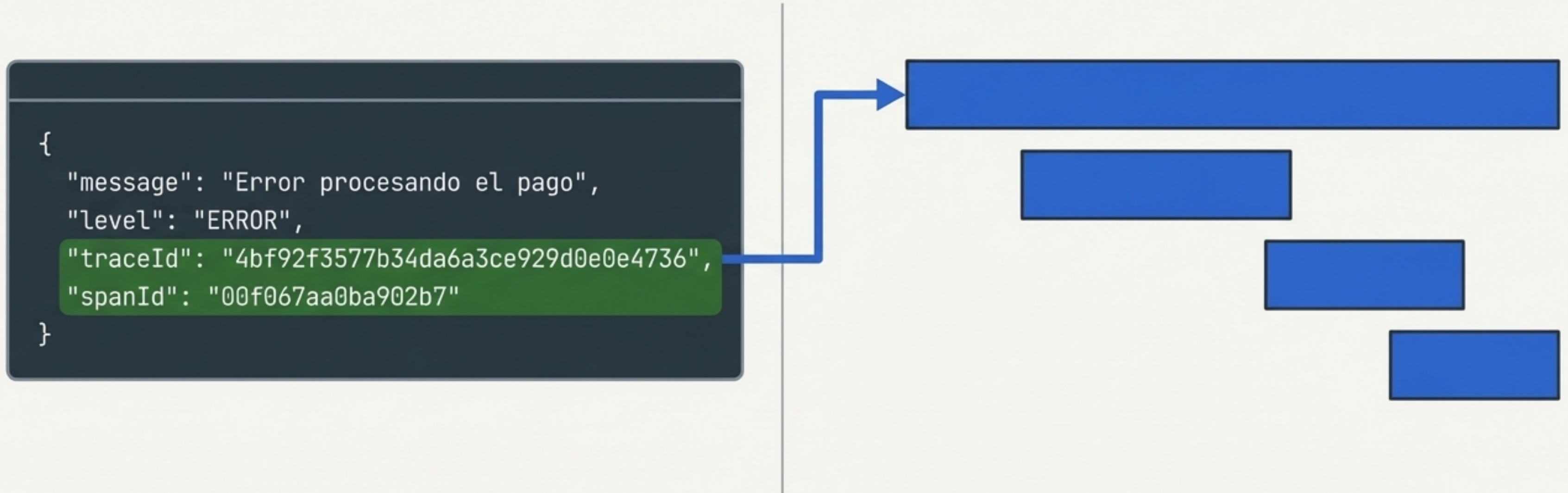
La telemetría cambia de ser un proceso basado en archivos (pull) a un modelo de red estandarizado (push).



```
management.opentelemetry.logging.export.otlp.endpoint=http://otel-collector:4318/v1/logs
```


El Clímax Analítico: Correlación Automática

Registro Estructurado + OpenTelemetry = Trazabilidad con cero código.



Descubre un error en los logs, haz clic en el `traceId` y visualiza instantáneamente la cascada de llamadas a microservicios que causó el fallo.

Decisiones de Arquitectura: Actuator vs. OTel

Ambos sistemas pueden coexistir, pero resuelven problemas diferentes en entornos de producción.

Spring Boot Actuator

Modelo:

Basado en Pull (Scraping)

Casos de Uso Ideales:

- Health checks (Sondas de Kubernetes)
- Recolección de métricas con Prometheus
- Exposición de entorno e información de la aplicación

OpenTelemetry Starter

Modelo:

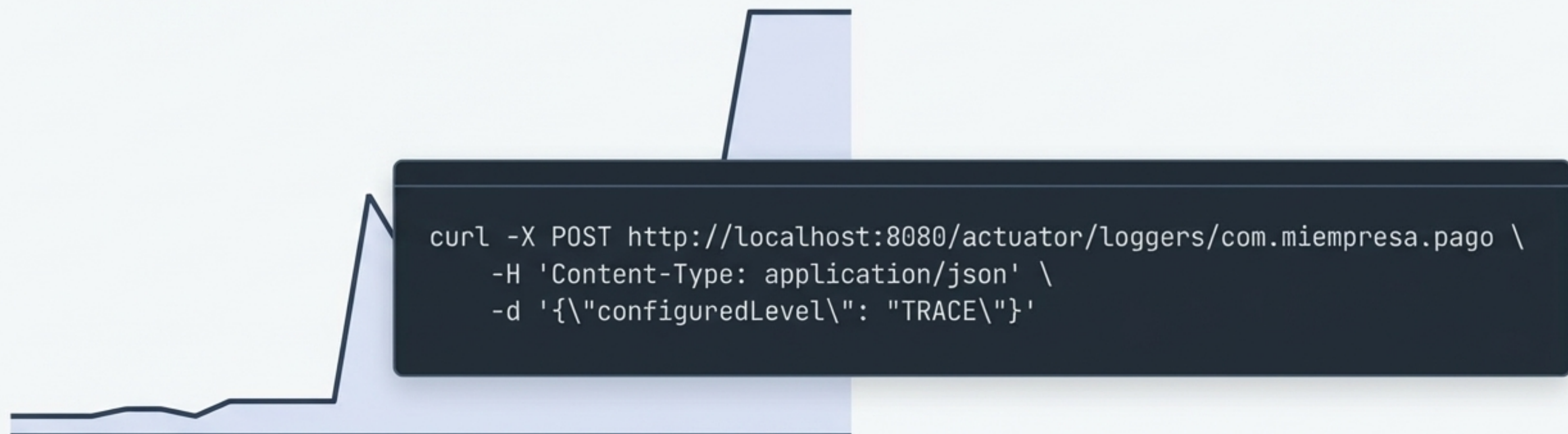
Basado en Push (Exportación OTLP)

Casos de Uso Ideales:

- Trazabilidad distribuida entre múltiples servicios
- Telemetría agnóstica del proveedor
- Correlación profunda de logs y trazas

Gestión de Niveles en Tiempo de Ejecución

Inspecciona e inyecta cambios de configuración en caliente sin reiniciar el servicio a través del endpoint /actuator/loggers.

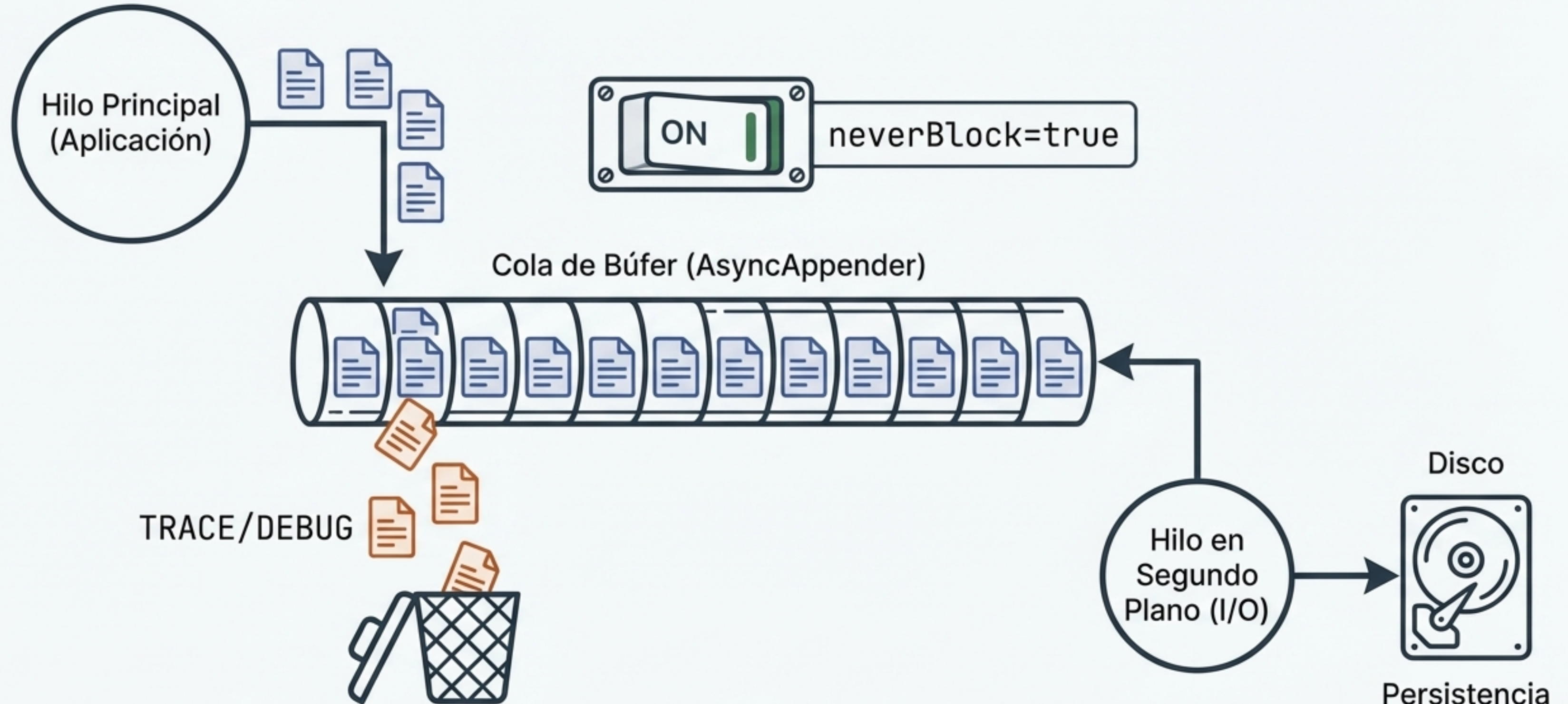


```
curl -X POST http://localhost:8080/actuator/loggers/com.miempresa.pago \  
-H 'Content-Type: application/json' \  
-d '{"configuredLevel": "TRACE"}'
```

Pasa de INFO a TRACE al instante para capturar un error escurridizo bajo demanda.

Rendimiento a Escala: Logging Asíncrono

Protección contra inanición de hilos (thread starvation) bajo cargas masivas.



Mejores Prácticas y Cumplimiento (Compliance)

Enmascaramiento de Datos Sensibles (PII)

password=MiSecreto123 → **Filtro Regex (%replace)** → password=****

Propagación de Contexto en Virtual Threads

```
public class ContextPropagatingTaskDecorator implements TaskDecorator {  
    ...  
}
```

Garantiza que el `traced` y el `MDC` no se pierdan al saltar entre hilos de plataforma y Virtual Threads.

Niveles por Entorno

Mantén INFO o WARN como línea base en producción. **Nunca dejes DEBUG habilitado globalmente para evitar impactos en el rendimiento y almacenamiento.**

La Nueva Frontera de la Observabilidad

Spring Boot 4 transforma el logging de 'escribir texto en disco' a un motor de telemetría de alto rendimiento.

Moderno

Java 25

Jakarta EE 11

Apache Commons 1.3

Estructurado

Jackson 3 Engine

JSON Nativo

Esquema ECS/GELF

Correlacionado

Micrometer Tracing

Inyección automática de MDC

Unificado

Protocolo OTLP

Exportación Push-based

Tu código cuenta la historia. Spring Boot 4 se asegura de que los sistemas puedan leerla.